

C++ Practice

Наследование

```
struct Base {  
    void print();  
};  
  
struct Derived : Base {  
    void makeAction();  
};  
  
int main() {  
    Derived object;  
    object.print();  
}
```

Инкапсуляция

Модификаторы доступа

```
struct Car {  
private:  
    int _wheels = 4;  
public:  
    int getWheels() { return _wheels; }  
};  
  
int main() {  
    Car car;  
    // std::cout << car._wheels; – нельзя получить доступ к приватному полю  
    std::cout << car.getWheels(); // можно вызвать публичный метод  
}
```

```
struct PublicByDefault {  
    void print(); // public  
private:
```

```
    int _x;
};

// ==

class PrivateByDefault {
    int _x; // private
public:
    void print();
};
```

```
class Base {
    int _x = 7;
public:
    int getX() { return _x; }
};

class PrivateDerived : private Base {
// private:
    // int getX();
public:
    void print() { std::cout << getX(); }
};

class PublicDerived : public Base {
// public:
    // int getX();
public:
    void print() { std::cout << getX(); }
}

class ProtectedDerived : protected Base {
// protected:
    // int getX();
public:
    void print() { std::cout << getX(); }
}

class ProtectedProtectedDerived : public ProtectedDerived {
// protected:
    // int getX();
// public:
    // void print() {
```

```

        getX();
    }
}

int main() {
    PrivateDerived privD;
    // privD.getX(); – нельзя вызвать приватный метод

    PublicDerived pubD;
    pubD.getX();

    ProtectedDerived protD;
    // protD.getX(); – protected методы могут вызывать только наследники
}

```

Полиморфизм

```

class Animal {
public:
    virtual void voice() const { std::cout << "Some Sound"; }
    // virtual void() { }
    virtual void print(int a) { }
    // virtual void voice() const = 0; // Что такое `= 0`?
    virtual ~Animal(); // Почему virtual? Нужен ли он здесь?
};

class Dog : public Animal {
public:
    void voice() const override
    { std::cout << "Woof" << std::endl; }

    void print() override { } // ошибка компиляции
};

class Cat : public Animal {
public:
    void voice() const override
    { std::cout << "Meow" << std::endl; }
};

int main() {
    Dog dog;
    Cat cat;

    std::vector<Animal*> animals{&dog, &cat};
}

```

```
    for (Animal* animal : animals) {  
        animal->voice(); // Some Sound  
    }  
}
```

Приведение типов

static_cast

```
int main() {  
    int intValue = 7;  
  
    // Неявное преобразование  
    double doubleImplicit = intValue; // 7.0  
  
    // Явное преобразование  
    double doubleExplicit = static_cast<double>(intValue); // 7.0  
}
```

dynamic_cast

```
class Animal {  
public:  
    virtual void voice() const { std::cout << ""; };  
    virtual ~Animal() { std::cout << "~Animal" << std::endl; }  
};  
  
class Dog : public Animal {  
public:  
    void voice() const override  
    { std::cout << "Woof" << std::endl; }  
  
    void sit() const  
    { std::cout << "Dog is sitting" << std::endl; }  
  
    ~Dog() { std::cout << "~Dog" << std::endl; }  
};  
  
class Cat : public Animal {  
public:  
    void voice() const override  
    { std::cout << "Meow" << std::endl; }  
  
    void catchMouse() const
```

```

        { std::cout << "Cat caught a mouse" << std::endl; }

~Cat() { std::cout << "~Cat" << std::endl; }
};

int main() {
    Animal* animalPtr = new Dog;
    animalPtr->voice(); // Woof

    // Хотим вызвать специфический метод собаки
    // animalPtr->sit(); – не получится

    // Если мы абсолютно уверены, что указатель указывает на собаку
    Dog* dogPtr = static_cast<Dog*>(animalPtr);
    dogPtr->sit(); // OK

    // Если самоуверенность нас подвела
    Cat* catPtr = static_cast<Cat*>(animalPtr);
    catPtr->catchMouse(); // Неопределенное поведение
    // Во время компиляции никаких ошибок не будет
    // g++ main.cpp -o main
    // никаких ошибок
    // ./main
    // Aborted Core dump

    delete animalPtr;
    animalPtr = nullptr;

    // Теперь мы не знаем, на какого наследника указывает указатель
    char userInput;
    std::cout << "(C)at or (D)og?" << std::endl;
    std::cin >> userInput;
    if (userInput == 'C') {
        animalPtr = new Cat;
    } else if (userInput == 'D') {
        animalPtr = new Dog;
    }

    Dog* dogPtr = dynamic_cast<Dog*>(animalPtr);
    if (dogPtr != nullptr) {
        dogPtr->sit();
    }

    Cat* catPtr = dynamic_cast<Cat*>(animalPtr);
    if (catPtr != nullptr) {
        catPtr->catchMouse();
    }
}

```

```

    }

    delete animalPtr;
    animalPtr = nullptr;
    dogPtr = nullptr;
    catPtr = nullptr;
}

```

`static_cast` выполняется во время компиляции (быстро)

`dynamic_cast` выполняется во время исполнения (медленно из-за проверок)

`static_cast` небезопасен - делает именно то, что говорят

`dynamic_cast` безопасен - в случае ошибки возвращает `nullptr` или бросает исключение

Не стоит злоупотреблять `dynamic_cast`

`dynamic_cast` обеспечивает безопасность, но не бесплатно

- Внутри он выполняет различные проверки, что приводит к лишним операциям(программа выполняется медленнее) и увеличенному расходу памяти
- Код, основанный на `if (cat)` - делай это, `else if (dog)` - делай другое ..., трудно поддерживать

```

// WalkService.h

class WalkService {
// std::vector<Animal*> _petsFriends{&cat, &mouse};
public:
    void goForAWalk(Animal* animalPtr) {
        // animalPtr->catchCat(...);
        // animalPtr->catchMouse(...);

        animalPtr->goForAWalk();

        /*if (Dog* dogPtr = dynamic_cast<Dog*>(animalPtr); dogPtr) {
            dogPtr->catchCat(_petsFriends[0]);
        } else if (Cat* catPtr = dynamic_cast<Cat*>(animalPtr); catPtr) {
            catPtr->catchMouse(_petsFriends[1]);
        } else if (...) {
            ...
        }*/
    }
}

```

```
// FeedService.h

class FeedService {
// std::vector<Meal*> _meal{&bone, &milk};
public:
    void feedPet(Animal* animalPtr) {
        if (Dog* dogPtr = dynamic_cast<Dog*>(animalPtr); dogPtr) {
            dogPtr->eatBone(_meal[0]);
        } else if (Cat* catPtr = dynamic_cast<Cat*>(animalPtr); catPtr) {
            catPtr->drinkMilk(_meal[1]);
        }
    }
}
}
```

```
// PlayService.h

class PlayService {
// std::vector<Toy*> _toys{&ball, &laser};
public:
    void playWithPet(Animal* animalPtr) {
        if (Dog* dogPtr = dynamic_cast<Dog*>(animalPtr); dogPtr) {
            dogPtr->runForBall(_toys[0]);
        } else if (Cat* catPtr = dynamic_cast<Cat*>(animalPtr); catPtr) {
            catPtr->runForLaser(_toys[1]);
        }
    }
}
}
```

Есть ли недостатки/проблемы у кода выше?

```
int main() {
    Animal* animalPtr = new Dog; // new Cat

    PlayService playService;
    playService.playWithPet(animalPtr);

    FeedService feedService;
    feedService.feedPet(animalPtr);

    WalkService walkService;
    walkService.goForAWalk(animalPtr);
}
```

```
    delete animalPtr;
}
```

```
int main() {
    Animal* animalPtr = new Dog; // new Cat

    animalPtr->goForAWalk();
    animalPtr->feed();
    animalPtr->play();

    delete animalPtr;
}
```

```
class Animal {
// std::vector<Animal*> _petsFriends{&cat, &mouse};
// std::vector<Meal*> _meal{&bone, &milk};
// std::vector<Toy*> _toys{&ball, &laser};

public:
    virtual void goForAWalk(Animal* friendPet) = 0;
    virtual void feed(Meal* meal) = 0;
    virtual void play(Toy* toy) = 0;
};

class Dog : public Animal {
public:
    void goForAWalk(Animal* friendPet) { catchCat(static_cast<Cat*>
(friendPet)); }
    void feed(Meal* meal) { eatBone(static_cast<Bone*>(meal)); }
    void play(Toy* toy) { runForBall(static_cast<Ball*>(toy)); }
private:
    void catchCat(Cat* cat);
    void eatBone(Bone* bone);
    void runForBall(Ball* ball);
};

// То же самое, но с другими приватными методами
// class Cat
```

1 Basic

```
class Animal {
public:
```



```

    virtual ~Animal() = default;
    virtual void goForAWalk(std::vector<Animal*>& friends) = 0;
    virtual void feed(std::vector<Meal*>& meals) = 0;
    virtual void play(std::vector<Toy*>& toys) = 0;
};

class Dog : public Animal {
public:
    void goForAWalk(std::vector<Animal*>& friends) override {
        catchCat(friends[0]);
    }
    void feed(std::vector<Meal*>& meals) override {
        eatBone(meals[0]);
    }
    void play(std::vector<Toy*>& toys) override {
        runForBall(toys[0]);
    }
};

class Cat : public Animal {
public:
    void goForAWalk(std::vector<Animal*>& friends) override {
        catchMouse(friends[1]);
    }
    void feed(std::vector<Meal*>& meals) override {
        drinkMilk(meals[1]);
    }
    void play(std::vector<Toy*>& toys) override {
        runForLaser(toys[1]);
    }
};

// Services become simple:
class WalkService {
    std::vector<Animal*> _petsFriends{&cat, &mouse};
public:
    void goForAWalk(Animal* animalPtr) {
        animalPtr->goForAWalk(_petsFriends);
    }
};

```

SOLID

Single Responsibility - принцип единственной ответственности

2 Visitor

Паттерн проектирования

```
class AnimalVisitor {
public:
    virtual void visit(Dog& dog) = 0;
    virtual void visit(Cat& cat) = 0;
    virtual ~AnimalVisitor() = default;
};

class WalkVisitor : public AnimalVisitor {
    std::vector<Animal*>& _friends;
public:
    WalkVisitor(std::vector<Animal*>& friends) : _friends(friends) {}

    void visit(Dog& dog) override {
        dog.catchCat(_friends[0]);
    }
    void visit(Cat& cat) override {
        cat.catchMouse(_friends[1]);
    }
};

class Animal {
public:
    virtual ~Animal() = default;
    virtual void accept(AnimalVisitor& visitor) = 0;
};

class Dog : public Animal {
public:
    void accept(AnimalVisitor& visitor) override {
        visitor.visit(*this);
    }
};

// Service:
class WalkService {
    std::vector<Animal*> _petsFriends{&cat, &mouse};
public:
    void goForAWalk(Animal* animalPtr) {
        WalkVisitor visitor(_petsFriends);
        animalPtr->accept(visitor);
    }
};
```

3 Performance

```
class Animal {
public:
    enum Type { DOG, CAT };
    virtual Type getType() const = 0;
    virtual ~Animal() = default;
};

class Dog : public Animal {
public:
    Type getType() const override { return DOG; }
};

// Service:
class WalkService {
    std::vector<Animal*> _petsFriends{&cat, &mouse};
public:
    void goForAWalk(Animal* animalPtr) {
        switch(animalPtr->getType()) {
            case Animal::DOG:
                static_cast<Dog*>(animalPtr)->catchCat(_petsFriends[0]);
                break;
            case Animal::CAT:
                static_cast<Cat*>(animalPtr)->catchMouse(_petsFriends[1]);
                break;
        }
    }
};
```

Исключения

```
struct NullDevideException {
    NullDevideException() {}

    NullDevideException(const NullDevideException&)
    { std::cout << "Copy Constructor"; }

    std::string what = "Division by zero is not allowed!";
};

int divide(int a, int b) {
```

```

    if (b == 0) {
        NullDevideException e;
        throw e;
    }
    return a / b;
}

int main() {
    std::cout << divide(10, 0);
    std::cout << "After division"; // Не выводится
}

```

Если не обработать исключение, то программа аварийно завершится

```

input
terminate called after throwing an instance of 'NullDevideException'

...Program finished with exit code 134
Press ENTER to exit console.

```

Обработка исключений

```

int main() {
    try {
        std::cout << divide(10, 0);
    } catch(NullDevideException& e) {
        std::cout << e.what << std::endl;
    } catch(std::string& e) {
        std::cout << e << std::endl;
    } catch(...) {
        std::cout << "Some exception thrown" << std::endl;
    }
    std::cout << "After division"; // Выводится
}

```

Наследование и исключения

```

struct BaseException {

```

```
};

struct DerivedException : BaseException {

};

void func() {
    throw BaseException();
}

int main() {
    try {
        func();
    } catch(DerivedException& e) {
        std::cout << "caught DerivedException" << std::endl;
    } catch(BaseException& e) {
        std::cout << "caught BaseException" << std::endl;
    } catch(...) {
        std::cout << "caught ..." << std::endl;
    }
}
```

! Выполняется первый подходящий catch

Assertion

```
#define NDEBUG // релизная сборка
// ассерты не выполняются

// #define NDEBUG – дебажная сборка
// ассерты выполняются

int divide(int a, int b) {
    assert(b != 0); // если b == 0, то ассерт выстрелит, программа
    завершится
    return a / b;
}

int main() {
    int c = 0;

    if (c != 0) {
        divide(10, c);
    }
}
```

```

int d = 0;

if (d != 0) {
    divide(10, d);
}

int e = 0;
if (e != 0) {
    divide(10, e);
}

std::cout << divide(10, 0) << std::endl;
std::cout << "After division"; // В режиме разработки не выводится
}

```

Дебажная сборка

Релизная сборка

```

input
a.out: main.cpp:13: int divide(int, int): Assertion `b != 0' failed.

...Program finished with exit code 134
Press ENTER to exit console.

```

Ассерты позволяют аварийно завершить программу, если произошло что-то неожиданное (разработчик допустил ошибку)

Задание

Необходимо постить контент в разные соцсети

Хочется, чтобы посты публиковались одновременно во все однотипные соцсети

Требования

- ☐ Пользователь может выбрать тип контента:
 - ☐ Пост в мессенджер (Телеграм, макС)
 - ☐ Длинное видео (RuTube)
 - ☐ Короткое видео (Вк.Клипс)
- ☐ Пользователь может выбрать соцсети, в которые планируется опубликовать контент (Телеграм, макС, RuTube, Вк.Клипс)
- ☐ (Опционально) Пользователь может выбрать аккаунт или канал для публикации (+1 балл)

- ☐ Пользователь может указать содержимое контента
 - ☐ Для постов в мессенджер:
 - ☐ Текст сообщения
 - ☐ Вложения
 - ☐ Изображения
 - ☐ Видео
 - ☐ Для длинных видео:
 - ☐ Видео
 - ☐ Описание к видео
 - ☐ Для коротких видео
 - ☐ Видео
 - ☐ Описание к видео
- ☐ (Опционально) Пользователь может указать запланированное время публикации (+1 балл)
- ☐ Пользователь может подтвердить публикацию
- ☐ (Опционально) Пользователь может посмотреть список или историю публикаций (+1 балл) (вывод с помощью перегрузки оператора <<)

Рекомендации по архитектуре

- ☐ Базовый класс `Content`
 - ☐ Наследник `PostMessage`
 - ☐ Хранит текст сообщения (`string`)
 - ☐ Хранит массив вложений (`std::vector<Attachment*>`)
 - ☐ Наследник `LongVideo`
 - ☐ Хранит текст описания (`string`)
 - ☐ Хранит видео (`Video*`)
 - ☐ Наследник `ShortVideo`
 - ☐ Хранит текст описания (`string`)
 - ☐ Хранит видео (`Video*`)
- ☐ Базовый класс `Attachment`
 - ☐ Хранит название файла ("image.png", "video.mp4")
 - ☐ Наследник `Image`
 - ☐ Хранит размер изображения (ширина, высота)
 - ☐ Наследник `Video`
 - ☐ Хранит продолжительность видео (в секундах)
- ☐ Структура `Date` (для запланированных публикаций)
- ☐ Базовый класс `Service`

- ☐ Метод `publish(Content*, Date publishDate)`
- ☐ Наследник `Telegram`
 - ☐ свой код для публикации
- ☐ Наследник `Max`
 - ☐ свой код для публикации
- ☐ Наследник `RuTube`
 - ☐ свой код для публикации
- ☐ Наследник `VKClips`
 - ☐ свой код для публикации
- ☐ Класс `Published` (подумать над названием) (хранит информацию об опубликованном контенте)
 - ☐ Хранит `Content*` - опубликованный контент
 - ☐ Хранит сервисы, куда был опубликован контент
 - ☐ Хранит дату публикации
 - ☐ Возвращается из метода `publish(...)`

Требования по реализации

- ☐ Грамотное распределение кода по файлам (`.cpp`, `.h`) (1 класс - 2 файла) (+1 балл)
- ☐ Методы публикации выбрасывают исключение в случае неудачи (+1 балл)

Возможности для расширения (+ баллы)

- ☐ Базовый класс `Publisher` - ответственен за публикацию в отдельные категории соц сетей
 - ☐ Имеет метод `Response publish(Content* content);`
 - ☐ Наследник `MessengerPublisher` (для постов в мессенджеры)
 - ☐ Хранит список мессенджеров в виде указателей на базовый класс
 - ☐ `std::vector<Messenger*> messengers;`
 - ☐ Наследники класса `Messenger`:
 - ☐ `Telegram`
 - ☐ `Max`
 - ☐ Имеет метод `Response publish(Post* post);`
 - ☐ Наследник `LongVideoPublisher` (для длинных видео)
 - ☐ Хранит список сервисов в виде указателей на базовый класс
 - ☐ `std::vector<LongVideoService*> services;`
 - ☐ Наследники класса `LongVideoService`:
 - ☐ `RuTube`
 - ☐ Имеет метод `Response publish(LongVideo* video);`

- ☐ Наследник ShortVideoPublisher (для коротких видео)
 - ☐ Хранит список сервисов в виде указателей на базовый класс
 - ☐ `std::vector<ShortVideoService*> services;`
 - ☐ Наследники класса ShortVideoService :
 - ☐ VKClips
 - ☐ Имеет метод `Response publish(ShortVideo* video);`
- ☐ Классы Messenger , LongVideoService , ShortVideoService - выполняют сами запросы к API соц сетей
 - ☐ Имеют конкретных наследников (Telegram, макс, RuTube, VKClips)
 - ☐ Имеют виртуальные методы `Response callApi(Request* req);`
- ☐ Класс Response
 - ☐ Имеет поле код статуса (int или enum или отдельная структура)
 - ☐ 200 (OK) - все хорошо
 - ☐ 400 - ошибка на нашей стороне (сделали неправильный запрос к API)
 - ☐ 500 - ошибка на стороне сервера (чей API вызвали)
 - ☐ Имеет какое-то содержимое (текст) (опционально)
- ☐ Класс Request
 - ☐ Хранит url и параметры/заголовки (какую-то информацию со скриншота ниже)

bash

```
curl -X POST "https://api.telegram.org/bot<YOUR_TOKEN>/sendMessage" \
  -H "Content-Type: application/json" \
  -d '{"chat_id": "12345678", "text": "Hello, this is a test message"}
```

Use code with caution. 

- ☐ Возможность редактировать опубликованный контент

README.md

Что сделали, на какую оценку
В идеале добавить скриншоты

Критерии оценивания

Базовая оценка: 4
Остальное - дополнительные баллы

Срок выполнения

24.05.2026 23:59 - если хотите комментарии

31.05.2026 23:59 - если не хотите комментарии

Если хотите комментариев, но не успеваете - сделайте промежуточный коммит с текущим кодом, откройте пул-реквест, доделанный код отправите новым коммитом.

Git

Merge Conflicts, проблемы с ключами

```
git checkout main (master) // сначала переключаемся на базовую ветку  
git checkout -b student/student-name/task-publisher // создаем новую ветку,  
отрастающую от базовой
```